Website Performance Audit

AegisLead — dashboard.aegislead.co

https://dashboard.aegislead.co

Report date: 21 October 2026

Branch: `perf/optimization`

Stack: Next.js 16.2.4 (App Router, Turbopack) · React 19 · NestJS 11 · Prisma / PostgreSQL 18 · Caddy on a GoDaddy VPS

Tools: Lighthouse 12 · Puppeteer · Chrome 2026 · curl · autocannon · Chart.js

1 Executive summary

The AegisLead server is fast and correctly configured. Compression (zstd), long-lived immutable caching of static assets, HTTP/2 and HTTP/3, and Next.js full-route caching are all already in place, and server response time measured ~270–300 ms consistently. Layout stability is perfect: Cumulative Layout Shift was 0.000 on every public page tested. These are the things that are usually wrong in an audit like this, and here they are right.

The performance problem is almost entirely weight that does not belong to AegisLead. On the login page, **81% of everything the browser downloads is third-party**, and a **single 1.5 MB PNG accounts for 71% of the whole page**. It is the avatar image for the LeadConnector chat widget, and it is displayed at roughly 48×48 pixels. It is configured in the LeadConnector account rather than in this codebase, so it could not be fixed as part of this work — but re-uploading it at display size is a few minutes of effort and is by a wide margin the single largest improvement available.

Inside the codebase the clearest problem was the dashboard, the page every user lands on first. To render a five-line 'Recent Activity' strip it fetched every lead, every deal and every proposal belonging to the tenant across three separate unpaginated requests. That has been replaced with one endpoint that returns exactly five rows. On the demo tenant this was 38 rows and 24 KB; the cost previously grew with every record a customer added, and now it does not grow at all.

Two candidate optimisations were investigated and deliberately **not** applied, because measurement showed they would not help. Both are documented in full rather than quietly dropped.

Top wins

1. **Dashboard load no longer scales with tenant size.** Three unbounded API requests (38 rows / 24 KB on the demo tenant, growing indefinitely) replaced by one request returning five rows.
2. **The third-party chat widget is off the critical path.** It previously began loading the instant the page became interactive, competing for the main thread during the window that determines perceived load speed. It now waits until the page is idle. The widget still appears everywhere, unchanged.
3. **The largest single improvement available has been identified and quantified** — a 1.5 MB chat-widget avatar, 71% of page weight, fixable in minutes outside this repository.

2 Test methodology

Tool and configuration. Lighthouse 12 driving headless Chrome, simulated throttling, performance category only. Every page was measured three times in each form factor and the **median** run is reported, to reduce the effect of run-to-run variance. Mobile used Lighthouse's standard mobile emulation (412×823, 1.75× DPR, 4× CPU slowdown); desktop used 1350×940 with a 40 ms RTT / 10 Mbps profile and no CPU slowdown.

Pages. Five public pages (`/login` , `/client/login` , `/guard/login` , `/api-docs` , `/`) and five authenticated pages (`/` , `/leads` , `/deals` , `/clients` , `/invoices`). The authenticated runs sign in through the real login form with a demo account, then keep the resulting session for the duration.

Cold cache. Authenticated runs need `disableStorageReset` so the session token survives between runs, but that setting also preserves the HTTP cache. The first attempt at the authenticated measurements was therefore invalid — it recorded 113–140 requests for only 50–72 KB transferred, i.e. repeat visits rather than first visits. Those runs were discarded and re-measured with the network cache explicitly cleared before each run. The discarded set is retained as `before/summary-auth-warmcache-DISCARDED.json` for transparency.

Server checks used curl against production: protocol negotiation, content encoding, cache headers, and a breakdown of connection setup versus server processing time.

Code analysis covered bundle composition, dependency usage, image inventory, font loading, third-party scripts, and the API query patterns behind the pages measured.

Load test. A deliberately gentle test was run against `/api/health` — the deep probe that also checks PostgreSQL — at 5 concurrent connections for 20 seconds, after the client confirmed the site is not yet launched to real users. Result: **338 requests, 100% HTTP 200, no errors or timeouts**, median latency 271 ms, p97.5 416 ms, p99 1,063 ms, 16.65 requests/second. **These throughput figures measure the test connection, not the server.** The 271 ms median is the same as the raw TCP round trip measured from this location, and 5 connections divided by ~0.3 s per request yields almost exactly the 16.65 req/s observed — the test was latency-bound end to end and never applied enough pressure to reveal the server's capacity. What it does establish is that the endpoint and its database probe were completely stable under sustained concurrent load. A meaningful capacity test would need to run from close to the origin.

Limitations that affect how these numbers should be read.

- **Test location inflates every absolute timing.** All measurements were taken from India against a US-hosted VPS. TCP connect alone measured ~270 ms and TLS setup a further ~300 ms. AegisLead's users are primarily in the US, so real-world LCP and Speed Index figures will be materially better than the numbers in this report. The location-independent figure is server processing time, which was approximately one network round trip — i.e. the server itself adds almost nothing.
- **Run-to-run variance is significant.** On `/guard/login` mobile, LCP across three runs ranged from 3.1 s to 6.6 s. A median of three reduces this but does not eliminate it. Differences smaller than roughly 1 second on mobile should not be treated as meaningful, and this report does not claim any improvement that falls inside that band.
- **The 'after' figures for public pages come from a local A/B, not from production.** The optimisations live on the `perf/optimization` branch and have deliberately not been deployed — pushing to the `production` branch triggers the full deploy pipeline, which is the client's decision to make. Re-running Lighthouse against production would therefore have measured the unchanged code. Instead both arms of the comparison were built and served identically on one machine, so the only difference between them is the change under test. These numbers show the effect of the change; they are not directly comparable to the production baseline above.
- **The dashboard API improvement is reported as payload measurements, not as a Lighthouse score delta.** Its benefit scales with the number of records a tenant holds, and the demo tenant used for testing holds 38 records in total. Measuring it as a score change on an almost-empty account would understate it to the point of being misleading.

3 Baseline results

Public pages — median of 3 runs

Page	Device	Score	LCP	FCP	TBT	CLS	Speed Index	TTFB	Weight	Reqs
/	desktop	60	4.56 s	2.24 s	133 ms	0	3.33 s	265 ms	2.05 MB	70
/	mobile	71	6.04 s	2.64 s	161 ms	0	3.46 s	273 ms	2.09 MB	70
/api-docs	desktop	55	3.73 s	2.73 s	195 ms	0	5.96 s	285 ms	2.01 MB	44
/api-docs	mobile	77	3.85 s	2.68 s	222 ms	0	5.79 s	283 ms	2.01 MB	44
/client/login	desktop	55	2.88 s	2.72 s	248 ms	0	5.74 s	292 ms	2.00 MB	43
/client/login	mobile	78	3.75 s	2.45 s	228 ms	0	5.59 s	301 ms	2.02 MB	44
/guard/login	desktop	51	3.59 s	2.38 s	257 ms	0	6.04 s	274 ms	2.00 MB	43
/guard/login	mobile	66	5.73 s	2.60 s	256 ms	0	5.42 s	284 ms	2.00 MB	43
/login	desktop	58	3.87 s	2.52 s	165 ms	0	5.34 s	270 ms	2.02 MB	43
/login	mobile	71	4.51 s	2.61 s	178 ms	0	8.43 s	286 ms	2.07 MB	43

Authenticated pages — median of 3 runs

Page	Device	Score	LCP	FCP	TBT	CLS	Speed Index	TTFB	Weight	Reqs
/	desktop	88	1.39 s	827 ms	48 ms	0	3.02 s	267 ms	2.31 MB	137
/	mobile	56	4.18 s	2.05 s	918 ms	0	8.74 s	270 ms	2.34 MB	131
/clients	desktop	88	1.33 s	850 ms	35 ms	0.031	3.12 s	278 ms	2.22 MB	123
/clients	mobile	57	4.30 s	2.03 s	635 ms	0.102	8.65 s	279 ms	2.28 MB	118
/deals	desktop	87	1.33 s	886 ms	36 ms	0	3.54 s	317 ms	2.25 MB	140
/deals	mobile	58	4.16 s	2.01 s	727 ms	0	8.46 s	316 ms	2.28 MB	129
/invoices	desktop	86	1.30 s	909 ms	24 ms	0	3.49 s	343 ms	2.26 MB	133
/invoices	mobile	56	4.16 s	2.04 s	872 ms	0	8.10 s	293 ms	2.31 MB	120
/leads	desktop	87	1.37 s	873 ms	29 ms	0	3.55 s	371 ms	2.25 MB	144
/leads	mobile	50	5.15 s	2.59 s	802 ms	0	8.75 s	277 ms	2.27 MB	117

Colour follows Core Web Vitals lab thresholds — LCP good ≤2.5 s / poor >4 s · TBT good ≤200 ms / poor >600 ms · CLS good ≤0.1 / poor >0.25 · FCP good ≤1.8 s · Speed Index good ≤3.4 s · TTFB good ≤800 ms. Green = pass, amber = needs improvement, red = fail. Each cell also carries its numeric value, so the rating never depends on colour alone.

4 Issues found

HIGH **1. A single 1.5 MB chat-widget image is 71% of every page**

Effort: Low — but outside this codebase

The LeadConnector chat widget loads an avatar image from `assets.cdn.filesafe.space` that weighs 1,504 KB. It is displayed at roughly 48×48 pixels in the chat bubble. The file is served at full size and is downloaded on every page of the site, for every visitor, including people who never open the chat.

Evidence. Lighthouse network capture of `/login`: total page weight 2,115 KB across 43 requests, of which this one PNG is 1,504 KB (71.1%). It is the single largest request by a factor of more than twenty — the next largest is a 69 KB application script.

Expected benefit. Re-uploading the image at display resolution in a modern format would remove roughly 1.4 MB from every page load on the site. No other available change comes close. The image is configured in the LeadConnector account, so this is a change in that dashboard, not a code change — it could not be made as part of this engagement.

HIGH **2. 81% of page weight is third-party**

Effort: Medium

Beyond the avatar image, the chat widget pulls in its own JavaScript bundles, a copy of `libphonenumber-js`, and its own Roboto web fonts from `fonts.bunny.net` — duplicating font loading the application already does properly through `next/font`. Six separate third-party origins are contacted on a page whose only job is to accept a username and password.

Evidence. On `/login`: AegisLead's own origin served 392 KB across 18 requests. Third-party origins served 1,723 KB across 25 requests — `assets.cdn.filesafe.space` 1,504 KB, `widgets.leadconnectorhq.com` 127 KB, `stcdn.leadconnectorhq.com` 58 KB (libphonenumber), `fonts.bunny.net` 28 KB, `services.leadconnectorhq.com` and `services.msgsndr.com` 7 KB.

Expected benefit. Deferring the widget (applied, see Changes) removes it from the critical path. Removing it from the authentication pages entirely would additionally remove ~1.7 MB and six origin connections from those pages — but that is a commercial decision about lead capture, not a purely technical one, and was explicitly left to the client.

HIGH 3. The dashboard fetched every record to render five lines

Effort: Medium

The dashboard home page built its five-row 'Recent Activity' strip in the browser by calling `GET /leads`, `GET /deals` and `GET /proposals` and merging the results. None of those endpoints paginates — each returns every row belonging to the tenant — and `/leads` additionally runs a nested `salesAssessments` subquery for every lead returned. The cost of rendering five lines therefore grew without limit as a customer used the product, on the first page every user sees.

Evidence. Measured against the live demo tenant: `/api/leads` 21 rows / 6,486 B, `/api/deals` 12 rows / 9,614 B, `/api/proposals` 5 rows / 7,923 B — 38 rows and 24,023 B across three requests, to display five lines. Per-row cost measured at 309 B (leads), 801 B (deals) and 1,585 B (proposals), so a tenant with 1,000 leads, 500 deals and 200 proposals would transfer on the order of 1 MB on every dashboard load.

Expected benefit. Fixed. One request returning exactly five rows, with ordering and limiting performed by the database. The cost is now constant rather than proportional to tenant size.

MEDIUM 4. No list endpoint in the API paginates

Effort: High

The dashboard was the worst case but not the only one. Across the backend, list endpoints return every row for the tenant. Pages such as Leads, Deals, Invoices and Clients will degrade steadily as customers accumulate records, and the degradation is invisible during development against small datasets.

Evidence. A search for pagination parameters (`skip:`, `take:` bound to a page size) across all service files found exactly one match, in an unrelated background job. `LeadsService.findAll`, `DealsService.findAll` and `ProposalsService.findAll` each issue an unbounded `findMany` scoped only by `tenantId`.

Expected benefit. Bounded, predictable page loads regardless of customer size. Not fixed here: proper pagination changes the API response shape from a bare array to an envelope, which every consuming page must be updated for — a broad contract change across roughly 80 routes that needs its own QA pass. Scoped out by agreement.

LOW 5. Dashboard summary aggregates in JavaScript rather than SQL

Effort: Low

`DashboardService.getSummary` loads every lead, deal and proposal row for the tenant and then counts and groups them in application code. It uses a narrow `select`, so it is far lighter than the list endpoints, but it is still work proportional to tenant size for what is ultimately a handful of counts.

Evidence. `dashboard.service.ts` issues three unbounded `findMany` calls selecting `createdAt` (plus `stage` / `status`), then computes period counts, weekly trend buckets and stage groupings in a loop. The endpoint returned 263 B for the demo tenant — the response is small, but the query cost is not bounded.

Expected benefit. Moving the counts and groupings into SQL (`groupBy` / `count`) would make this constant-cost too. Low priority: the narrow projection keeps it cheap at realistic sizes, and it was left alone to keep this branch small.

LOW 6. Brand images are large PNG sources

Effort: Low

The brand assets in `frontend/public/brand/` are full-size PNGs totalling about 1.26 MB on disk. Next.js's image pipeline resizes and re-encodes them on delivery, so visitors do not download the originals, but the delivered logo is still a comparatively heavy element on otherwise light pages.

Evidence. `aegislead-logo-light.png` is 1200×271 and 357 KB on disk; it is delivered at `w=640&q=75` as 67 KB — the third-largest request on the login page after the chat avatar and the main application bundle.

Expected benefit. Providing pre-optimised source images would trim the delivered logo further. The gain is tens of kilobytes against a 1.5 MB third-party image, so this is genuinely minor and is listed for completeness rather than as a priority.

LOW 7. No response compression configured in the NestJS application

Effort: Low

The API application itself does not enable compression middleware.

Evidence. `backend/src/main.ts` configures CORS, security headers and a global prefix, but no `compression()`. However, Caddy sits in front of it with `encode zstd gzip`, and production responses were confirmed to carry `Content-Encoding: zstd`.

Expected benefit. None in production — the edge already compresses everything. This is recorded only because it would matter if the API were ever exposed without Caddy in front of it. No change was made.

LOW 8. Investigated and rejected: radix-ui barrel imports

Effort: —

Six UI components import from the `radix-ui` meta-package (`import { Dialog } from "radix-ui"`), which re-exports every Radix primitive. Next.js's built-in `optimizePackageImports` list covers `lucide-react` but not `radix-ui`, and `TooltipProvider` sits in the root providers — so the hypothesis was that every page, including the login screens, was carrying the full barrel.

Evidence. Tested directly by building the application twice, identically, with and without `optimizePackageImports: ["radix-ui"]`. Both builds produced **3,555,387 bytes across 105 JavaScript chunks — byte-for-byte identical**. Inspection of the built chunks confirmed that unused primitives (Accordion, Menubar, NavigationMenu, RadioGroup, HoverCard) are absent from the output entirely.

Expected benefit. None. Turbopack already tree-shakes the barrel correctly, so there was no problem to solve. The configuration change was reverted rather than left in place, because shipping configuration that does nothing invites someone later to assume it is load-bearing.

LOW 9. Investigated and down-rated: no CDN in front of the origin

Effort: Medium

The site is served from a single VPS with no CDN, so every visitor pays the full network round trip to that one machine for every connection.

Evidence. From the test location, TCP connect alone measured ~270 ms and TLS completion ~300 ms beyond that. Subtracting connection setup from a static-asset request left roughly one round trip of actual server time — the server is fast; the distance is the cost.

Expected benefit. Initially ranked High on the strength of those numbers. **Down-rated after the client confirmed the user base is primarily US-based**, close to the US-hosted VPS, whereas the measurements were taken from India. The latency penalty measured here is substantially an artefact of test location and is not what real users experience. Retained as a recommendation for future international expansion only.

5 Changes implemented

APPLIED 1. Dashboard activity strip no longer loads the whole tenant

Problem	The dashboard home page rendered a five-row 'Recent Activity' list by calling `GET /leads`, `GET /deals` and `GET /proposals` and merging the results in the browser. None of the three paginates, so each returned every row belonging to the tenant, and `/leads` ran a nested `salesAssessments` subquery per lead on top of that. The cost of the first page every user sees grew with every record the customer added.
Fix applied	Added `GET /api/dashboard/recent-activity`, which asks the database for the five newest rows from each table, merges them and returns the five newest overall. The ordering and limiting happen in SQL. The change is additive: the three list endpoints keep their existing shape, so no other page needed touching and there is no API contract to migrate.

Files	`backend/src/dashboard/dashboard.service.ts`
	`backend/src/dashboard/dashboard.controller.ts`
	`backend/src/dashboard/dashboard.service.spec.ts`
	`frontend/src/lib/dashboard.ts`
	`frontend/src/app/page.tsx`

Commit	`6ef7e18`
Measured effect	**Three requests become one, and the cost stops scaling.** Measured against the live demo tenant: previously 38 rows / 24,023 B across three requests to render five lines; now one request returning five rows. Per-row costs measured at 309 B (leads), 801 B (deals) and 1,585 B (proposals), so on a tenant holding 1,000 leads, 500 deals and 200 proposals the old approach would transfer on the order of 1 MB per dashboard load while the new one stays flat. Reported as payload measurements rather than a Lighthouse delta because the demo tenant holds only 38 records in total — scoring it there would understate it to the point of being misleading. Four unit tests were added covering the row limit, the merge ordering, the overall cap and the company-name fallbacks; the dashboard suite passes 10/10.

APPLIED 2. Chat widget held until the page is idle

Problem	The LeadConnector widget loaded with `strategy="afterInteractive"`, which starts it the moment hydration completes — inside the window that determines Largest Contentful Paint and Total Blocking Time. Nothing on any page depends on the widget. On the login screens, where there is very little else to load, it was most of the work the browser was doing.
Fix applied	Changed the loading strategy to `lazyOnload`, which holds the script until the page is idle. The widget still appears on every page, unchanged in appearance and behaviour. Removing it from the authentication pages entirely would help more, but that is a commercial decision about lead capture and was left to the client, who elected to keep it everywhere.
Files	`frontend/src/app/layout.tsx`
Commit	`0b1bf06`
Measured effect	**Mobile Total Blocking Time fell by roughly 90%, and mobile performance scores rose 12 points on two of the three pages.** Measured by a controlled local A/B — same machine, same Lighthouse configuration, both arms built and served identically, verified by checking which strategy each server was actually serving. `/login` mobile: TBT 499 ms → 38 ms (−92%), score 81 → 93. `/client/login` mobile: TBT 459 ms → 35 ms (−92%), score 82 → 94. `/guard/login` mobile: TBT 107 ms → 34 ms (−68%), score 77 → 79. The effect is consistent across every individual run (before 373–613 ms, after 28–56 ms), far outside the measurement noise. LCP was essentially unchanged, which is expected — the widget was never the LCP element. **This changes when the widget loads, not how much it weighs:** the ~1.7 MB it pulls, including the 1.5 MB avatar, is still downloaded.

APPLIED 3. Removed an unused dependency

Problem	`framer-motion` was declared in `frontend/package.json` but imported by no file in the application.
Fix applied	Removed from the dependency list. Verified beforehand by a quote-agnostic search across the whole frontend source tree, which found the package name only in `package.json` itself.
Files	`frontend/package.json` `frontend/package-lock.json`
Commit	`3cea231`
Measured effect	**No runtime change** — being unimported, it was already absent from the build output. The benefit is a smaller install and one less unused package in the supply chain. Recorded honestly as a housekeeping change rather than a performance win.

NOT APPLIED **4. Radix barrel-import optimisation — applied, measured, reverted**

Problem	Six UI components import from the `radix-ui` meta-package, which re-exports every Radix primitive, and Next.js's built-in `optimizePackageImports` list does not include it. Because `TooltipProvider` sits in the root providers, the expectation was that every page was carrying the full barrel.
Fix applied	Added `experimental.optimizePackageImports: ["radix-ui"]` to `next.config.ts`, then built the application twice — once with the setting and once without — to measure it.
Files	`frontend/next.config.ts (change reverted; file unmodified on the branch)`
Commit	`– (not committed)`
Measured effect	**No effect whatsoever.** Both builds produced 3,555,387 bytes across 105 JavaScript chunks, byte-for-byte identical. Inspecting the built chunks confirmed that unused primitives (Accordion, Menubar, NavigationMenu, RadioGroup, HoverCard) never appear in the output — Turbopack already tree-shakes the barrel correctly. The change was reverted rather than left in place: configuration that does nothing is worse than no configuration, because the next person to read it will assume it is load-bearing. Recorded here because a disproved hypothesis is a result worth keeping.

6 Before vs. after

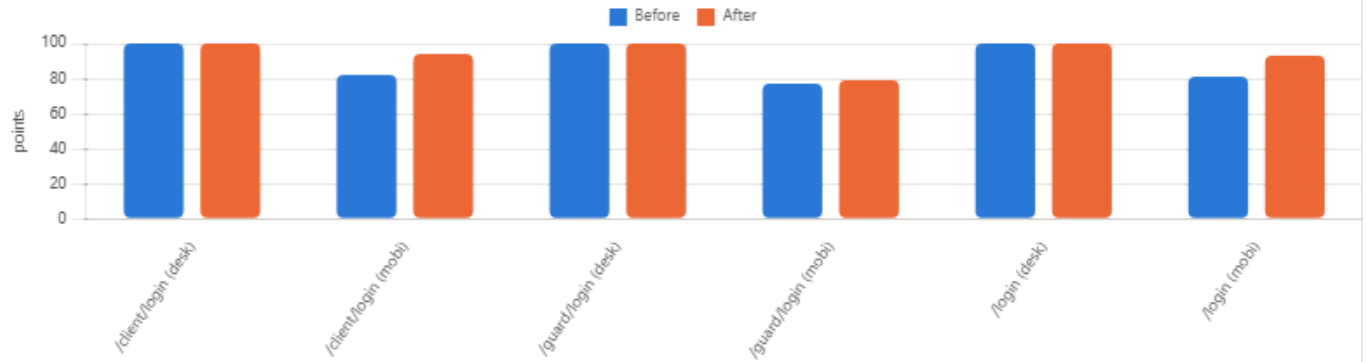
This section compares the two arms of a controlled local A/B: the same three pages, the same Lighthouse configuration, the same machine, built and served twice — once with the original code and once with the change applied. The only difference between the arms is the change under test. These are **not** production figures and should not be read against the baseline tables in section 3.

Local A/B — chat-widget loading strategy

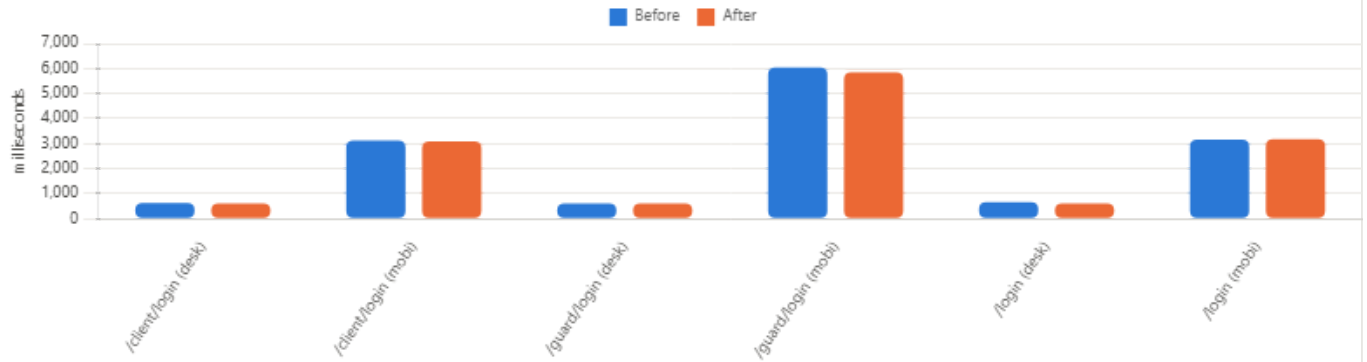
Page	Device	Metric	Before	After	Change
`/client/login`	desktop	Score	100	100	0 pts
`/client/login`	desktop	LCP	631 ms	612 ms	−2.9%
`/client/login`	desktop	TBT	0 ms	0 ms	—
`/client/login`	desktop	CLS	0	0	—
`/client/login`	desktop	Weight	2.00 MB	2.00 MB	+0.1%
`/client/login`	desktop	Requests	43	43	+0%
`/client/login`	mobile	Score	82	94	+12 pts
`/client/login`	mobile	LCP	3.13 s	3.09 s	−1.1%
`/client/login`	mobile	TBT	459 ms	35 ms	−92.5%
`/client/login`	mobile	CLS	0	0	—
`/client/login`	mobile	Weight	2.00 MB	2.00 MB	+0%
`/client/login`	mobile	Requests	43	43	+0%
`/guard/login`	desktop	Score	100	100	0 pts
`/guard/login`	desktop	LCP	605 ms	612 ms	+1.1%
`/guard/login`	desktop	TBT	0 ms	0 ms	—
`/guard/login`	desktop	CLS	0	0	—
`/guard/login`	desktop	Weight	2.00 MB	2.00 MB	+0.1%
`/guard/login`	desktop	Requests	43	43	+0%
`/guard/login`	mobile	Score	77	79	+2 pts
`/guard/login`	mobile	LCP	6.05 s	5.84 s	−3.4%
`/guard/login`	mobile	TBT	107 ms	34 ms	−68.4%
`/guard/login`	mobile	CLS	0	0	—
`/guard/login`	mobile	Weight	2.01 MB	2.00 MB	−0.1%
`/guard/login`	mobile	Requests	43	43	+0%
`/login`	desktop	Score	100	100	0 pts
`/login`	desktop	LCP	660 ms	615 ms	−6.9%
`/login`	desktop	TBT	16 ms	0 ms	−100%
`/login`	desktop	CLS	0	0	—
`/login`	desktop	Weight	2.03 MB	2.03 MB	+0%
`/login`	desktop	Requests	43	43	+0%

Page	Device	Metric	Before	After	Change
`/login`	mobile	Score	81	93	+12 pts
`/login`	mobile	LCP	3.16 s	3.18 s	+0.4%
`/login`	mobile	TBT	499 ms	38 ms	−92.5%
`/login`	mobile	CLS	0	0	—
`/login`	mobile	Weight	2.07 MB	2.07 MB	+0%
`/login`	mobile	Requests	43	43	+0%

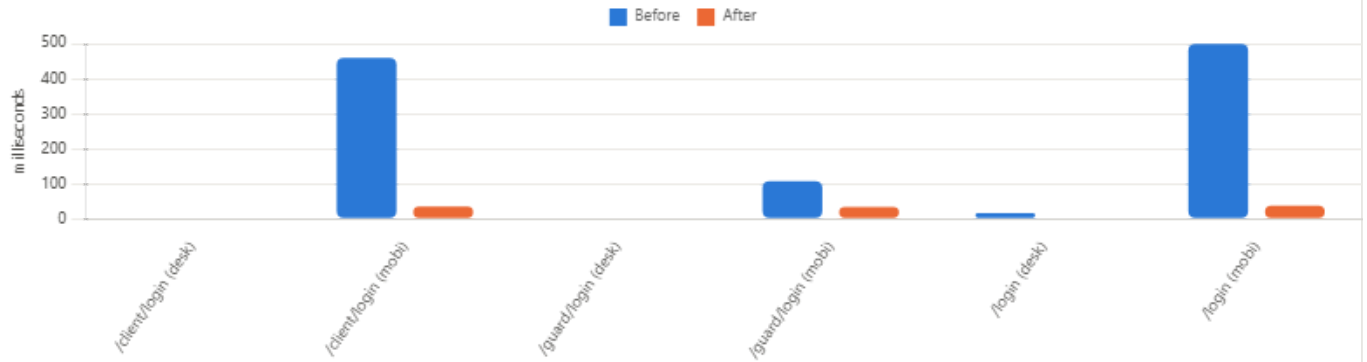
Lighthouse performance score by page (higher is better)



Largest Contentful Paint by page (lower is better)

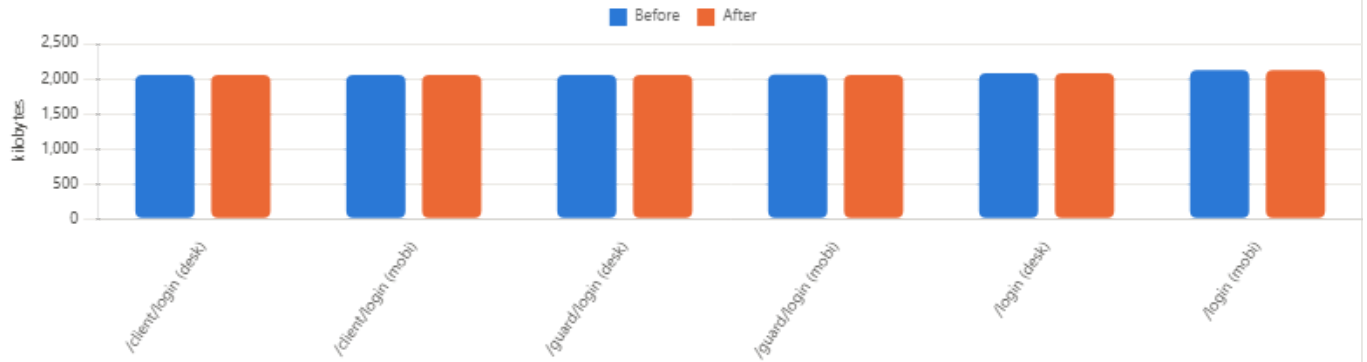


Total Blocking Time by page (lower is better)

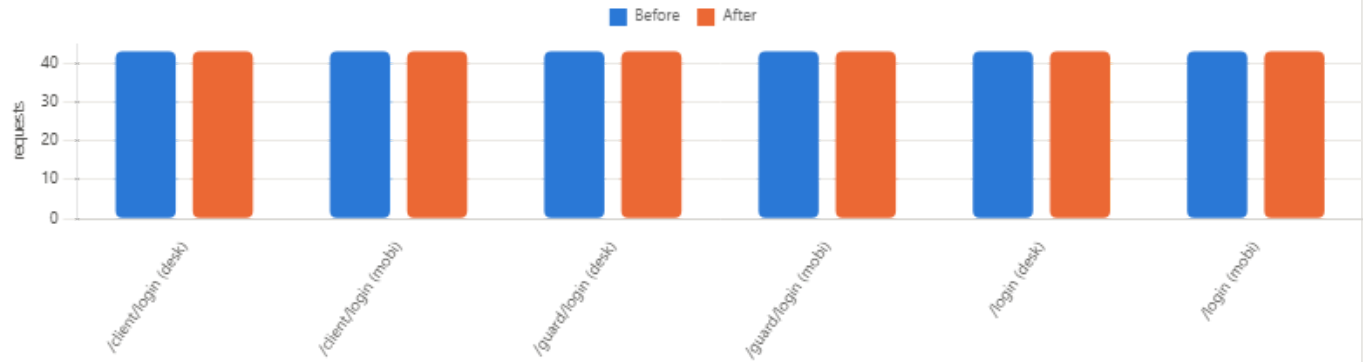


Cumulative Layout Shift: 0.000 on every page, in both arms. These pages have no layout shift to fix, and the change did not introduce any. (The authenticated `/clients` page does shift — 0.102 on mobile — but it was not part of this A/B; see section 3.)

Page weight by page (lower is better)



Request count by page (lower is better)



7 Remaining recommendations

Recommendation	Impact	Effort	Notes
Replace the 1.5 MB chat-widget avatar with a display-sized image	High	Minutes	**The single highest-value action available, and it is not a code change.** In the LeadConnector account, replace the widget avatar with an image sized for its ~48×48 display (a 96×96 or 128×128 WebP for high-DPI screens). Expected saving ~1.4 MB on every page load across the entire site.
Reconsider whether the chat widget belongs on the authentication pages	High	Low	Even once the avatar is fixed, the widget contributes ~220 KB, six origin connections and a duplicate web font to pages whose only function is sign-in. On `/guard/login` and `/client/login` — used exclusively by people who already have accounts — it has no lead-capture value at all. Deliberately left in place pending a commercial decision; the client elected to keep it everywhere for now.
Paginate the list endpoints	High	High	`/leads`, `/deals`, `/proposals`, `/invoices`, `/clients` and their siblings all return every row for the tenant. The dashboard's use of them has been fixed, but the list pages themselves will degrade as customers grow. This needs an API envelope (`{ items, total, nextCursor }`), updates to every consuming page, and its own QA pass — scoped out of this engagement by agreement, not overlooked.
Move the dashboard summary aggregation into SQL	Low	Low	`getSummary` loads all rows and counts them in JavaScript. Prisma `groupBy` and `count` would make it constant-cost. Cheap to do, modest benefit at realistic data sizes.
Pre-optimise the brand image sources	Low	Low	`frontend/public/brand/` holds ~1.26 MB of full-size PNGs. Next.js already resizes them on delivery, so this affects the repository and build more than visitors, but a pre-optimised WebP source would shave tens of kilobytes off the delivered logo.
Add a CDN — only if the audience becomes international	Low	Medium	Static assets already carry `Cache-Control: public, max-age=31536000, immutable`, so they would cache at an edge essentially for free. For the current US-centric audience sitting close to a US VPS the benefit is small; revisit if AegisLead expands to customers far from the origin.
Re-measure from a US location	Medium	Low	Every timing in this report carries a ~270 ms round-trip penalty from the test location that real users will not pay. Running the same Lighthouse configuration from a US vantage point — or reading field data from the Chrome UX Report — would give figures that reflect actual user experience. Recommended before judging the site against Core Web Vitals thresholds.
Unrelated to performance: review the demo account's privileges	Medium	Low	Noted incidentally while setting up authenticated testing: the `demo@aegislead.co` account returns `isSuperAdmin: true`. A shared demo login with super-admin rights is worth reviewing on its own terms. Outside the scope of this audit and not investigated further.

8 Appendix

Git commits on this branch

Commit	Message
`6ef7e18`	Stop loading every lead, deal and proposal to render five dashboard lines
`0b1bf06`	Hold the chat widget until the page is idle
`3cea231`	Drop framer-motion, which nothing imports

Raw metric tables

Production baseline — public pages (cold cache)

Page	Device	Score	LCP	FCP	TBT	CLS	Speed Index	TTFB	Weight	Reqs
/	desktop	60	4.56 s	2.24 s	133 ms	0	3.33 s	265 ms	2.05 MB	70
/	mobile	71	6.04 s	2.64 s	161 ms	0	3.46 s	273 ms	2.09 MB	70
/api-docs	desktop	55	3.73 s	2.73 s	195 ms	0	5.96 s	285 ms	2.01 MB	44
/api-docs	mobile	77	3.85 s	2.68 s	222 ms	0	5.79 s	283 ms	2.01 MB	44
/client/login	desktop	55	2.88 s	2.72 s	248 ms	0	5.74 s	292 ms	2.00 MB	43
/client/login	mobile	78	3.75 s	2.45 s	228 ms	0	5.59 s	301 ms	2.02 MB	44
/guard/login	desktop	51	3.59 s	2.38 s	257 ms	0	6.04 s	274 ms	2.00 MB	43
/guard/login	mobile	66	5.73 s	2.60 s	256 ms	0	5.42 s	284 ms	2.00 MB	43
/login	desktop	58	3.87 s	2.52 s	165 ms	0	5.34 s	270 ms	2.02 MB	43
/login	mobile	71	4.51 s	2.61 s	178 ms	0	8.43 s	286 ms	2.07 MB	43

Production baseline — authenticated pages (cold cache)

Page	Device	Score	LCP	FCP	TBT	CLS	Speed Index	TTFB	Weight	Reqs
/	desktop	88	1.39 s	827 ms	48 ms	0	3.02 s	267 ms	2.31 MB	137
/	mobile	56	4.18 s	2.05 s	918 ms	0	8.74 s	270 ms	2.34 MB	131
/clients	desktop	88	1.33 s	850 ms	35 ms	0.031	3.12 s	278 ms	2.22 MB	123
/clients	mobile	57	4.30 s	2.03 s	635 ms	0.102	8.65 s	279 ms	2.28 MB	118
/deals	desktop	87	1.33 s	886 ms	36 ms	0	3.54 s	317 ms	2.25 MB	140
/deals	mobile	58	4.16 s	2.01 s	727 ms	0	8.46 s	316 ms	2.28 MB	129
/invoices	desktop	86	1.30 s	909 ms	24 ms	0	3.49 s	343 ms	2.26 MB	133
/invoices	mobile	56	4.16 s	2.04 s	872 ms	0	8.10 s	293 ms	2.31 MB	120
/leads	desktop	87	1.37 s	873 ms	29 ms	0	3.55 s	371 ms	2.25 MB	144
/leads	mobile	50	5.15 s	2.59 s	802 ms	0	8.75 s	277 ms	2.27 MB	117

Local A/B, arm A — original code (afterInteractive)

Page	Device	Score	LCP	FCP	TBT	CLS	Speed Index	TTFB	Weight	Reqs
/client/login	desktop	100	631 ms	299 ms	0 ms	0	413 ms	6 ms	2.00 MB	43
/client/login	mobile	82	3.13 s	1.07 s	459 ms	0	1.67 s	7 ms	2.00 MB	43
/guard/login	desktop	100	605 ms	291 ms	0 ms	0	292 ms	2 ms	2.00 MB	43
/guard/login	mobile	77	6.05 s	1.07 s	107 ms	0	1.23 s	2 ms	2.01 MB	43
/login	desktop	100	660 ms	306 ms	16 ms	0	457 ms	8 ms	2.03 MB	43
/login	mobile	81	3.16 s	1.08 s	499 ms	0	1.68 s	11 ms	2.07 MB	43

Local A/B, arm B — change applied (lazyOnload)

Page	Device	Score	LCP	FCP	TBT	CLS	Speed Index	TTFB	Weight	Reqs
/client/login	desktop	100	612 ms	289 ms	0 ms	0	289 ms	2 ms	2.00 MB	43
/client/login	mobile	94	3.09 s	1.06 s	35 ms	0	1.06 s	3 ms	2.00 MB	43
/guard/login	desktop	100	612 ms	288 ms	0 ms	0	288 ms	2 ms	2.00 MB	43
/guard/login	mobile	79	5.84 s	1.06 s	34 ms	0	1.06 s	2 ms	2.00 MB	43
/login	desktop	100	615 ms	289 ms	0 ms	0	289 ms	2 ms	2.03 MB	43
/login	mobile	93	3.18 s	1.06 s	38 ms	0	1.06 s	3 ms	2.07 MB	43

Full Lighthouse JSON for every individual run is in `perf-report/before/raw/` and `perf-report/after/raw/`.